

AI-SOC / SecOps Agents

Agentic AI + Cybersecurity - AI-Native Textbook Skeleton (REQUIRED)

Purpose: A Hackathon-1 style skeleton for an AI-native textbook and production-grade system. This version marks the “industry extras” as **REQUIRED**: K8s hardening, CI/CD, RAG evaluation, observability, supply-chain security, cost controls, and OpenAI SDK integration.

One-line Goal

AI is no longer limited to the computer screen - it now works as an **agent** inside SOC/SecOps to investigate, decide, and safely respond to security incidents.

Date: January 04, 2026

Authoring Method: Spec-driven + AI-assisted writing, then published using Docusaurus.

Table of Contents

- 1. Overview and Audience
- 2. Core Deliverables (Book + Embedded AI Teacher)
- 3. Mandatory Platform & Architecture Requirements (Industry-level)
- 4. Modules and Chapter Map (4 Modules)
- 5. Capstone Project
- 6. Learning Outcomes
- 7. Weekly Breakdown (13 weeks)
- 8. Assessments
- 9. Production Deployment Plan (Local and Cloud)
- 10. Security, Reliability, and Cost Requirements (REQUIRED)
- 11. Final Deliverables Checklist (REQUIRED)

1. Overview and Audience

This textbook teaches how to build an **AI-SOC** system where agentic AI behaves like a SOC analyst: triaging alerts, enriching context, forming hypotheses, recommending response actions, and generating incident reports. The book is published as a Docusaurus site and includes an embedded RAG chatbot that answers using the book content and selected text.

Target Audience

- SOC Analyst (L1/L2), Blue Team, DFIR learners
- Cybersecurity students and professionals moving into AI-native security operations
- Developers and AI engineers building security automation products
- Freelancers/consultants who want an industry-grade demo system

2. Core Deliverables (Book + Embedded AI Teacher)

Deliverable A - Docusaurus Textbook

- Complete chapter-based textbook written in Markdown and published with Docusaurus
- Clean navigation: sidebar, next/previous, search, and consistent styling
- Public deployment of the book site (GitHub Pages or equivalent static hosting)

Deliverable B - Embedded RAG Chatbot (AI Teacher inside the Book)

- An embedded chatbot UI available directly within the book website
- RAG pipeline that retrieves relevant sections from the book content stored in Qdrant
- Selected-text Q/A: if the user highlights text on a page, the chatbot prioritizes that selection in the answer
- Conversation persistence stored in a database (Neon Postgres) for session continuity
- Chatbot LLM integration uses the **OpenAI SDK** (required)

3. Mandatory Platform & Architecture Requirements (Industry-level)

All items in this section are **REQUIRED** (not optional). They ensure the solution is industry-grade and cloud-deployable.

Data and Storage (REQUIRED)

- Relational database: **Neon Postgres** for users, auth/session state, chat history, audit logs
- Vector database: **Qdrant** for embeddings and retrieval over the textbook + playbooks

LLM and Agent Runtime (REQUIRED)

- LLM integration via **OpenAI SDK** for chatbot responses and agent orchestration
- Strict guardrails: allow-list tool calls, approval gates for disruptive actions, and full audit logging

Containerization and Local Kubernetes (REQUIRED)

- **Docker** - all services run as container images
- **Minikube** - local Kubernetes cluster for development and testing
- **Helm** - packaging and deploying services via charts

Microservices + Dapr Runtime (REQUIRED)

- **Microservices architecture** - separate services for book site, chatbot API, ingestion/embedding, etc.
- **Dapr** - service invocation, pub/sub, secrets/config integration, resiliency patterns

Cloud Deployment + Operations (REQUIRED)

- Primary cloud target: **Google Cloud** (GKE for Kubernetes)
- CI/CD pipeline (GitHub Actions) for build, test, image push, and Helm deploy
- Observability stack (logs/metrics/traces) with SLO-style alerting rules

4. Modules and Chapter Map (4 Modules)

The book uses four main modules. Chapters are practical and aligned to building the capstone system.

Module 1 - SOC and Cybersecurity Foundations (Ground Reality)

- 1. SOC mission: logs -> alerts -> incidents -> response
- 2. Logs 101: authentication, web, endpoint, network
- 3. SIEM fundamentals: parsing, correlation, severity, triage
- 4. Threat intelligence basics: IoCs vs TTPs, context, sources
- 5. Evidence and timeline thinking: build the incident story
- 6. Common incident types: phishing, brute-force, malware beaconing, suspicious logins

Module 2 - Agentic AI Foundations (The Agent Brain)

- 1. Chatbot vs Agent: what changes when AI can act
- 2. Tool calling fundamentals: safe functions and structured outputs
- 3. Agent loops: plan -> act -> observe -> verify
- 4. Memory: short-term context vs long-term RAG
- 5. Guardrails: allow-list actions, policy checks, human approval gates
- 6. Reliability: retries, idempotency, timeouts, and safe failure

Module 3 - AI-SOC Workflows (Investigation to Response)

- 1. Triage workflow: alert intake -> context -> priority -> next actions
- 2. Enrichment: asset, user, IP reputation, geo, historical activity
- 3. Detection engineering: rules, false positives, tuning
- 4. Response playbooks: isolate host, disable user, block IOC, reset credentials
- 5. Case management: tickets, handoffs, executive summaries
- 6. Human-in-the-loop operations: approval before disruptive actions

Module 4 - Securing AI Systems (AI Security and Adversarial Thinking)

- 1. Prompt injection, data leakage, and tool abuse risks
- 2. Secure RAG: source filtering, citations, and data boundaries
- 3. Hardening agents: sandboxed tools, least privilege, auditing
- 4. Red teaming the agent: adversarial prompts and misuse scenarios
- 5. Compliance and governance: logging, retention, and access control

5. Capstone Project

Build an industry-style **AI-SOC Agent System** that demonstrates end-to-end incident handling using microservices, Dapr, Neon Postgres, Qdrant RAG, and a Docusaurus-based textbook UI.

Capstone System Components (REQUIRED)

- Book Site Service (Docusaurus): hosts the textbook and embeds the chatbot widget
- Chatbot API Service (FastAPI): orchestrates agent responses, policies, and tool calls via OpenAI SDK
- Ingestion/Embedding Service: converts textbook + playbooks into embeddings and stores them in Qdrant
- Identity/Session & Audit Service: user auth/session state + chat persistence + audit logs in Neon Postgres
- Observability Service/Stack: centralized logs/metrics/traces + dashboards + alerting

Capstone Behaviors (must work)

- User pastes an alert: agent produces triage, severity, rationale, and next steps
- User highlights text on a book page: agent explains the selected text first (selected-text priority)
- Agent retrieves relevant book sections from Qdrant and uses them in the final answer (grounded responses)
- Agent produces a final incident report: timeline, findings, impact, remediation, and prevention steps
- All actions are safe: disruptive recommendations require approval gates and are logged

6. Learning Outcomes

- Understand SOC operations and how incidents are investigated end-to-end
- Design and build agentic AI workflows for triage, enrichment, investigation, and response
- Implement RAG using Qdrant to keep answers grounded in the textbook and playbooks
- Engineer safe tool-calling agents with guardrails, auditing, and approval gates
- Deploy an industry-grade microservices system using Docker, Minikube, Helm, and Dapr
- Ship to cloud (GKE) using CI/CD and observability practices

7. Weekly Breakdown (13 weeks)

Week	Focus
Week 1	Introduction: AI as a SOC worker, incident lifecycle, and agentic security thinking.
Week 2	Logs and evidence: authentication, endpoint, network; build timelines.
Week 3	SIEM fundamentals: correlation thinking, severity, false positives.
Week 4	SOC playbooks: triage + reporting; clean incident summaries.
Week 5	Agent foundations: tools, structured outputs, safe action design.
Week 6	OpenAI SDK integration: agent loop + policy checks + safe tool calling.
Week 7	RAG build: chunking, embeddings, Qdrant indexing, retrieval testing.
Week 8	RAG quality & evaluation: grounded-answer checks + hallucination control.
Week 9	Microservices decomposition + Neon schemas + audit logging.
Week 10	Local K8s: Docker images, Helm charts, Minikube deployment.
Week 11	Dapr integration: invocation, pub/sub, secrets/config, resiliency.
Week 12	Cloud deployment: GKE + CI/CD (GitHub Actions) + observability.
Week 13	Security hardening + supply-chain security + cost controls + final demo.

8. Assessments

Assessment measures security understanding, agent engineering, and production readiness.

- Module quizzes: SOC concepts, agent patterns, safe operations, governance
- Mini Project 1: alert triage + incident summary (structured report format)
- Mini Project 2: tool-calling agent with guardrails (approval gate + audit log)
- Mini Project 3: RAG evaluation suite (retrieval metrics + grounded-answer checks)
- Mini Project 4: observability setup (logs/metrics/traces + basic SLO alerts)
- Mini Project 5: supply-chain security (SBOM + dependency and image scanning)
- Final Capstone: AI-SOC Agent System demo + final documentation/report

9. Production Deployment Plan (Local and Cloud)

Local (Developer) Deployment - REQUIRED

- Build Docker images for each microservice
- Deploy to Minikube using Helm charts
- Run Dapr sidecars for service invocation and pub/sub
- Verify end-to-end: book site -> chatbot -> Qdrant -> Neon Postgres

Cloud (Industry) Deployment - REQUIRED

- Kubernetes cluster on Google Cloud (GKE)
- Secrets/config managed securely and integrated via Dapr
- CI/CD via GitHub Actions: build -> test -> docker push -> helm deploy
- Centralized observability: logs, metrics, traces; alerting for failures and latency
- Public URL for the Docusaurus book site with embedded AI teacher

10. Security, Reliability, and Cost Requirements (REQUIRED)

This section converts previously 'bonus' items into mandatory requirements.

10.1 Kubernetes Security Hardening (REQUIRED)

- RBAC: least-privilege roles and service accounts
- NetworkPolicies: restrict east-west traffic between microservices
- Pod Security: baseline/restricted posture where possible; avoid privileged containers
- Secrets strategy: encrypted secrets, rotation, and least-privilege access (integrated via Dapr where applicable)

10.2 CI/CD Example (GitHub Actions) - REQUIRED

- Pipeline stages: build -> test -> docker push -> helm deploy
- Use environment separation: dev/staging/prod (at minimum dev + prod)
- Gate releases on tests and image scanning results

10.3 RAG Quality & Evaluation - REQUIRED

- Retrieval metrics: hit rate/recall on a small curated Q/A set
- Grounded answers checks: ensure answers cite retrieved chunks and stay within sources
- Hallucination control: fallback behavior, refusal rules, and confidence thresholds

10.4 Observability Stack Detail - REQUIRED

- Logs, metrics, traces across services (correlation by request id/trace id)
- Dashboards for latency, error rate, throughput, and dependency health
- Alerting rules (SLO-style): error budget burn, high latency, elevated 5xx, queue backlogs

10.5 Supply-Chain Security - REQUIRED

- Image scanning in CI (fail on critical vulnerabilities)
- SBOM generation for releases
- Dependency checks (lockfiles, vulnerability scanning, and update strategy)

10.6 Cost + Rate Limiting - REQUIRED

- Request rate limits per user/session (protect LLM and backend)
- Caching: cache repeated retrieval results and safe responses where applicable
- Token budgets: per-request token caps and monthly budget monitoring (cloud friendly)

11. Final Deliverables Checklist (REQUIRED)

Use this as the final submission checklist. All items are mandatory.

- Docusaurus textbook repository with clean structure and complete chapters
- Public book deployment URL (GitHub Pages or equivalent)
- Embedded chatbot inside the book UI using OpenAI SDK (selected-text Q/A works)
- Qdrant collection populated with book and playbook embeddings
- Neon Postgres schema for users, sessions, chat history, and audit logs
- Docker images for all services (microservices)
- Helm charts for deploying all services
- Minikube demo: local Kubernetes deployment verified
- Dapr integration configured for services
- CI/CD GitHub Actions: build -> test -> docker push -> helm deploy
- Observability: logs/metrics/traces + SLO-style alerts configured
- K8s hardening: RBAC + NetworkPolicies + Pod Security + secrets strategy
- Supply-chain security: image scanning + SBOM + dependency checks
- Cost controls: rate limiting + caching + token budgets
- Cloud deployment plan documented (GKE recommended) and runnable
- Capstone demo scenario + final incident report example